

Before you run AI-generated SQL.

A safety cheat sheet for queries that can change, expose, or break data.

- Read before you run
- Command types
- Preview first
- Blast radius
- Transactions
- Least privilege
- Parameterized inputs
- Prod checklist

Sagar Rathkanthiwar
Data & AI Professional

10 slides · save for later →

Plausible is not the same as safe

AI writes SQL that **parses, runs, and looks right**. What it cannot know is which environment you're connected to, which table is the real one, or what the business considers safe to touch.

- **Wrong environment** — the statement was fine; the connection was production.
- **Wrong object** — `orders` vs `orders_archive` vs `orders_v2`. The model guessed.
- **Wrong scope** — a filter that matches 4 rows in your head and 400,000 in the table.

STOP

Read the whole statement before it touches a connection.

CHECK

Which database? Which objects? How many rows? Reversible?

RUN

Only once all three answers are known — not assumed.

Stop → Check → Run. Ten seconds of reading is cheaper than any restore you will ever do.

Know what the first word can do to you

SELECT

Reads only. Can still be slow or expose data.

INSERT

Adds rows. Can duplicate or violate constraints.

UPDATE

Overwrites values. The old ones are gone.

DELETE

Removes rows. Logged and usually reversible inside a transaction.

MERGE

Insert + update + delete in one. Hardest to review.

CREATE

Adds an object. Low risk — unless it replaces one.

ALTER

Changes structure. Can lock tables and break downstream code.

DROP

Destroys the object and its data. Often not reversible.

TRUNCATE

Empties a table fast. Frequently non-transactional.

DDL (CREATE / ALTER / DROP / TRUNCATE) behaves very differently by engine. PostgreSQL can roll most of it back; MySQL and Oracle commit it implicitly the moment it runs. Verify yours — don't assume.

Turn every write into a read — first

Before running a write, run the **exact same filter** as a `SELECT`. Same table, same predicate, no edits.

```
-- 1. COUNT the blast radius
SELECT COUNT(*) FROM orders
WHERE status = 'pending' AND created_at < '2026-01-01';

-- 2. LOOK at real rows before trusting the filter
SELECT order_id, status, created_at FROM orders
WHERE status = 'pending' AND created_at < '2026-01-01'
LIMIT 20;

-- 3. Only now, with the SAME predicate, write
UPDATE orders SET status = 'expired'
WHERE status = 'pending' AND created_at < '2026-01-01';
```

If the count surprises you, **the filter is wrong — not the data**. Some engines also let you see exactly which rows a write touched: `RETURNING` (PostgreSQL), `OUTPUT` (SQL Server).

A missing **WHERE** updates everything

```
-- The most expensive typo in SQL:  
UPDATE customers SET tier = 'gold'; -- every customer. all of them.  
DELETE FROM orders;                -- the whole table.  
  
-- Constrain by key, by partition, by date, by batch:  
UPDATE customers SET tier = 'gold'  
WHERE customer_id IN (101, 102, 103);  
  
DELETE FROM events  
WHERE event_date >= '2026-01-01' AND event_date < '2026-02-01';
```

- **Confirm the environment first** — `SELECT CURRENT_DATABASE(), CURRENT_USER;` costs nothing.
- **Filter on the partition key** on large or columnar warehouses — it caps cost and damage at once.
- **Delete in batches** instead of one giant statement; long writes hold locks and stall production.
- **Engine note:** `LIMIT` on `UPDATE` / `DELETE` works in MySQL, but **not** in PostgreSQL or most warehouses — bound the predicate instead.

Ask of every write: **“If this matched ten times more rows than I expect, what breaks?”**

Give yourself an undo button

```
BEGIN;                                -- open the transaction

UPDATE orders SET status = 'expired'
WHERE status = 'pending' AND created_at < '2026-01-01';

-- VALIDATE inside the transaction, before you commit:
SELECT status, COUNT(*) FROM orders GROUP BY status;

COMMIT;    -- numbers look right? make it permanent
ROLLBACK;  -- numbers look wrong? undo everything since BEGIN
```

- **Verify your engine's behaviour.** Autocommit is on by default in many clients, and some tools commit each statement as it runs.
- **DDL is the exception.** MySQL and Oracle commit `CREATE` / `ALTER` / `DROP` implicitly — there's nothing left to roll back.
- **Don't leave it open.** An uncommitted transaction holds locks and can block everyone else.

A transaction is not a substitute for a backup. **It protects the next five minutes, not the next five days.**

The query ran fine. That's not the risk.

A `SELECT` can't corrupt a table — but it can move regulated data somewhere it was never approved to go.

```
-- RISKY: pulls every column, including ones you never asked for
SELECT * FROM customers; -- email, dob, national_id, card_last4...

-- SAFER: name the columns, mask what you don't need in full
SELECT customer_id,
       LEFT(email, 2) || '***' AS email_hint,
       signup_date
FROM customers
WHERE signup_date >= '2026-01-01';
```

- **Least privilege** — analysts read from views, not base tables. Write access is granted, never assumed.
- **Never paste real PII, secrets or credentials into an AI prompt.** Share the schema, not the rows.
- **Exports are where data leaks** — a CSV in a downloads folder has left every control you have.

`SELECT *` in a shared notebook or on a screen share is a **disclosure decision**, not a shortcut.

Never build SQL by gluing strings together

When AI generates **application** code, string-concatenated SQL is the classic injection hole: user input becomes executable syntax.

```
# UNSAFE - the input becomes part of the statement
q = "SELECT * FROM users WHERE email = '" + user_input + "'"
#   user_input = "x' OR '1'='1"           -> returns every user
#   user_input = "x'; DROP TABLE users;--" -> ends very badly

# SAFE - the value is sent separately from the SQL text
cur.execute(
    "SELECT id, email FROM users WHERE email = %s",
    (user_input,)
)
```

- Placeholders differ by driver (`%s` , `?` , `:name`) — the principle doesn't: **data is data, not syntax**.
- Table and column names can't be parameterized — validate them against an **allow-list**, never against user input.

Ad-hoc analysis and shipped code are different risk classes. **Anything that accepts outside input must be parameterized** — no escaping by hand.

The 8-point pre-flight check

Run this before any statement touches a production system — whoever, or whatever, wrote it.

- 1 Environment** — confirmed dev, staging or prod. Say it out loud.
- 2 Backup** — a recent snapshot, and a restore path you've tested.
- 3 Peer review** — a second human has read the statement.
- 4 Expected rows** — you predicted the count before running it.
- 5 Query cost** — bytes scanned, locks, runtime, impact on others.
- 6 Permissions** — least privilege; not connected as a superuser.
- 7 Logging** — who ran what, when, against which objects.
- 8 Rollback plan** — written down before the statement executes.

If you can't answer **all eight**, the query isn't ready — no matter how good it looks.

THE TAKEAWAY

AI can write the statement.

You own the consequence.

- A model has no idea which connection is open, who is on call, or what that table means to the business.
- Permissions, business impact, security and production safety are **human accountabilities**. They don't transfer.
- Safe SQL isn't a syntax skill. It's a **habit** — read, preview, constrain, wrap, verify.

What's the most dangerous SQL command you've seen run in production?

Share it in the comments — someone is about to run it today.

Sagar Rathkanthiwar · [Data & AI Professional](#) · [follow for more field guides](#)

Note on dialects: examples use generic SQL unless labelled otherwise. Transaction and DDL behaviour, `LIMIT` on writes, masking functions and driver placeholders vary across PostgreSQL, MySQL, SQL Server, Oracle, Snowflake and BigQuery — verify against your engine's documentation before running anything destructive.